

Activity TA 1

NAME:- ARADHYA DIKE

ROLL NO,:- A-16

Github https://github.com/Aradhyadike/ANNDL/A16_TA1.pdf

Stock Price Prediction using RNN and LSTM

Objective: Develop a Recurrent Neural Network (RNN) with Long Short-Term Memory (LSTM) units in Python to predict future stock prices based on historical data.

Dataset: We will use a subset of the S&P 500 stock data, specifically focusing on a company like Apple nc. (AAPL) for the last 5 years (2019-2024).

1. Data Preparation

Instructions: Choose any company listed in the S&P 500 with at least 5 years of historical data. Perform preprocessing steps, including feature selection, normalization, and scaling, to prepare the data for use in an RNN model.

```
# Install necessary libraries if not already installed in Google Colab/Local environment
!pip install pandas numpy scikit-learn tensorflow matplotlib plotly yfinance

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (5.24.1)
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (0.2.66)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.1)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.80.0)
Requirement already satisfied: tensorboard~>2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly) (9.1.4)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.0.13)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.9.6)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.4.7)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.0.5)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.13.5)
Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.15.0)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (15.0.1)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.44.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfinance) (2.7.0)
```

Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2.0.0)
 Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2024.2.2)
 Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (13.9.4)
 Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
 Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.0)
 Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10.1)
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.2.3)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
import plotly.graph_objects as go
import datetime
import yfinance as yf

# Choose a specific company (e.g., Apple Inc. - AAPL)
ticker = "AAPL"

# Filter for the last 5 years (2019-2024)
start_date = "2019-01-01"
end_date = "2024-01-01"

# Download data using yfinance
df = yf.download(ticker, start=start_date, end=end_date)

# Check if columns are MultiIndex and flatten them if they are
# This often happens with yfinance output, where 'Close' might be under a 'Price' level.
if isinstance(df.columns, pd.MultiIndex):
    # Change to get_level_values(0) to correctly extract metric names like 'Close', 'High', etc.
    df.columns = df.columns.get_level_values(0)

# Drop any rows with NaN values in 'Close'
df = df.dropna(subset=['Close'])

# Check if the DataFrame is empty after filtering
if df.empty:
    print(f"Warning: No data found for {ticker} within the range {start_date} to {end_date} after dropping NaN values.")
    print("Please check the ticker symbol or date range.")
else:
    print(f"Data for {ticker} from {start_date} to {end_date} loaded.")
df.head()
```

/tmp/ipykernel_27632/1126478617.py:17: FutureWarning:

YF.download() has changed argument auto_adjust default to True

[*****100%*****] 1 of 1 completedData for AAPL from 2019-01-01 to 2024-01-01 loaded.

	Price	Close	High	Low	Open	Volume
Date						
2019-01-02	37.503731	37.724594	36.627408	36.784150	148158800	
2019-01-03	33.768078	34.606402	33.722955	34.193175	365248800	
2019-01-04	35.209606	35.278479	34.150423	34.323786	234428400	
2019-01-07	35.131241	35.344980	34.649145	35.314106	219111200	
2019-01-08	35.800957	36.055068	35.271365	35.518348	164101200	

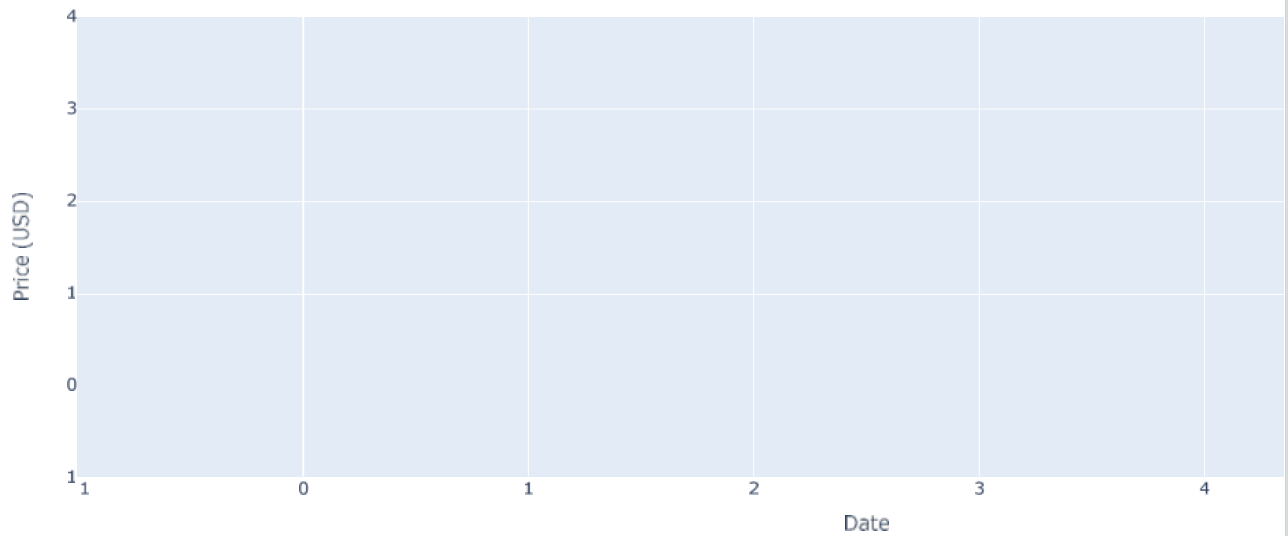
Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Visualizing using Candlestick Chart

fig = go.Figure(data=[go.Candlestick(x=df.index,
    open=df['Open'],
    high=df['High'],
    low=df['Low'],
    close=df['Close'])])

fig.update_layout(title=f'{ticker} Stock Price (2019-2024)', xaxis_title='Date', yaxis_title='Price (USD)')
fig.show()
```

AAPL Stock Price (2019-2024)



```
# Feature Selection: We will use the 'Close' price for prediction
data = df.filter(['Close'])
dataset = data.values

# Calculate the training data length (80% of the data)
training_data_len = int(np.ceil(len(dataset) * .8))

# Normalization and Scaling
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(dataset)

print(f"Shape of scaled data: {scaled_data.shape}")
```

Shape of scaled data: (1258, 1)

```
# Create the training data set
# Create the scaled training data set
train_data = scaled_data[0:int(training_data_len), :]

# Split the data into x_train and y_train data sets
x_train = []
y_train = []
sequence_length = 60 # Use past 60 days to predict the next day

for i in range(sequence_length, len(train_data)):
    x_train.append(train_data[i-sequence_length:i, 0])
    y_train.append(train_data[i, 0])

# Convert the x_train and y_train to numpy arrays
x_train, y_train = np.array(x_train), np.array(y_train)

# Reshape the data for LSTM (samples, time steps, features)
x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))
print(f"x_train shape: {x_train.shape}")
```

x_train shape: (947, 60, 1)

2. Model Development

Instructions: Construct an RNN model using libraries like TensorFlow or PyTorch. Incorporate LSTM units to address the vanishing gradient problem and improve memory retention across time steps.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout
```

```
# Build the LSTM model
model = Sequential()
model.add(LSTM(128, return_sequences=True, input_shape=(x_train.shape[1], 1)))
model.add(Dropout(0.2)) # Prevent overfitting
model.add(LSTM(64, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(25))
model.add(Dense(1)) # Output layer for predicting the next closing price

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning:

Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 128)	66,560
dropout (Dropout)	(None, 60, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 25)	1,625
dense_1 (Dense)	(None, 1)	26

Total params: 117,619 (459.45 KB)

Trainable params: 117,619 (459.45 KB)

3. Training

Instructions: Train the RNN model on the prepared dataset, optimizing the loss function and choosing an appropriate optimizer to enhance model performance.

```
# Compile the model
# We use the Adam optimizer and Mean Squared Error (MSE) loss function
model.compile(optimizer='adam', loss='mean_squared_error')

# Train the model
# Note: In a real-world scenario you may want to try different hyperparameters (batch_size, epochs)
history = model.fit(x_train, y_train, batch_size=32, epochs=20, validation_split=0.1)
```

```
Epoch 1/20
27/27 ————— 10s 203ms/step   loss: 0.0304   val_loss: 0.0025
Epoch 2/20
27/27 ————— 3s 111ms/step   loss: 0.0032   val_loss: 0.0012
Epoch 3/20
27/27 ————— 3s 112ms/step   loss: 0.0030   val_loss: 0.0020
Epoch 4/20
27/27 ————— 7s 172ms/step   loss: 0.0025   val_loss: 0.0012
Epoch 5/20
27/27 ————— 3s 110ms/step   loss: 0.0019   val_loss: 0.0012
Epoch 6/20
27/27 ————— 4s 136ms/step   loss: 0.0022   val_loss: 0.0013
Epoch 7/20
27/27 ————— 6s 173ms/step   loss: 0.0023   val_loss: 0.0011
Epoch 8/20
27/27 ————— 3s 109ms/step   loss: 0.0021   val_loss: 0.0011
Epoch 9/20
27/27 ————— 5s 109ms/step   loss: 0.0022   val_loss: 0.0020
Epoch 10/20
27/27 ————— 4s 163ms/step   loss: 0.0019   val_loss: 0.0027
Epoch 11/20
27/27 ————— 3s 120ms/step   loss: 0.0020   val_loss: 0.0015
Epoch 12/20
27/27 ————— 5s 110ms/step   loss: 0.0020   val_loss: 0.0011
Epoch 13/20
27/27 ————— 3s 117ms/step   loss: 0.0017   val_loss: 0.0011
Epoch 14/20
27/27 ————— 5s 109ms/step   loss: 0.0018   val_loss: 0.0017
Epoch 15/20
27/27 ————— 3s 108ms/step   loss: 0.0017   val_loss: 0.0010
Epoch 16/20
```

```

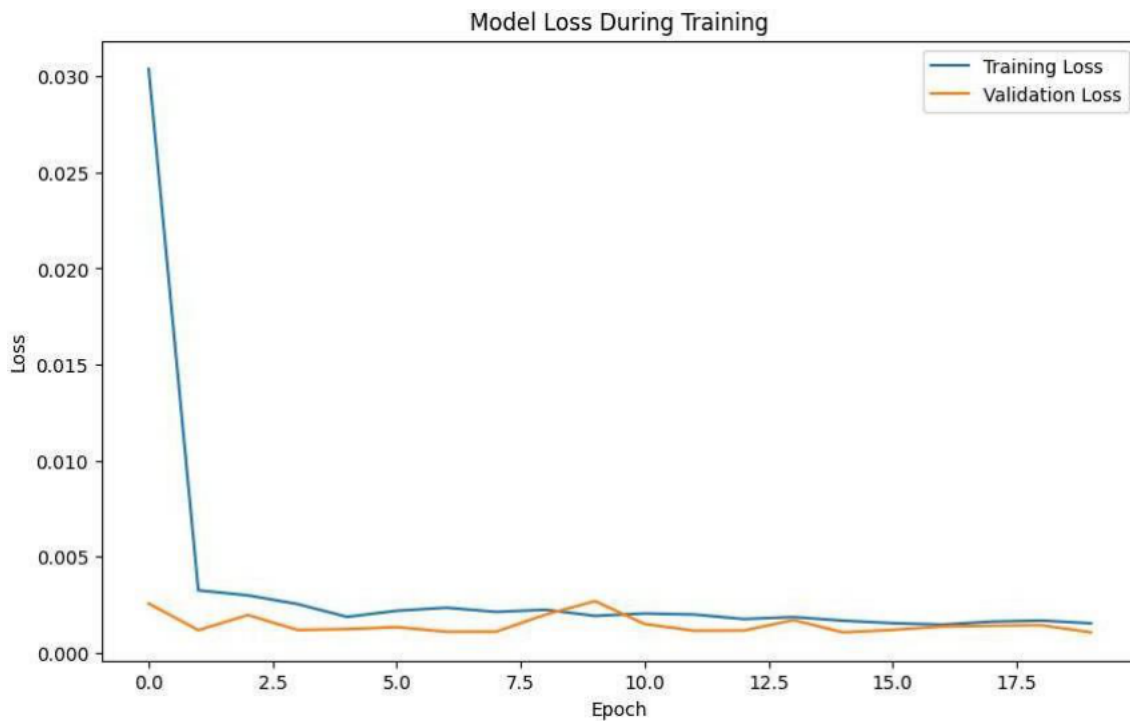
27/27 ----- 3s 112ms/step - loss: 0.0015 - val_loss: 0.0012
Epoch 17/20
27/27 ----- 5s 173ms/step - loss: 0.0015 - val_loss: 0.0014
Epoch 18/20
27/27 ----- 3s 110ms/step - loss: 0.0016 - val_loss: 0.0014
Epoch 19/20
27/27 ----- 3s 109ms/step - loss: 0.0017 - val_loss: 0.0014
Epoch 20/20
27/27 ----- 3s 110ms/step - loss: 0.0015 - val_loss: 0.0010

```

```

# Visualize training and validation loss
plt.figure(figsize=(10,6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend()
plt.show()

```



4. Prediction and Evaluation

Instructions: Enable the model to forecast future stock prices, starting from a given initial stock price input. Ensure the RNN mode efficiently learns from the selected stock price data and accurately forecasts future trends. Evaluate performance using MAE, RMSE, and MAPE.

```

# Create the testing data set
test_data = scaled_data[training_data_len - sequence_length:, :]

x_test = []
y_test = dataset[training_data_len:, :]

for i in range(sequence_length, len(test_data)):
    x_test.append(test_data[i-sequence_length:i, 0])

# Convert the data to a numpy array
x_test = np.array(x_test)

# Reshape the data
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))

# Get the model's predicted price values
predictions = model.predict(x_test)

```



```
predictions = scaler.inverse_transform(predictions) # Unscale to original values
```

8/8  1s 75ms/step

```
# Evaluation Metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error
import warnings
warnings.filterwarnings("ignore")

# Calculate RMSE (Root Mean Squared Error)
rmse = np.sqrt(np.mean(((predictions - y_test) ** 2)))

# Calculate MAE (Mean Absolute Error)
mae = mean_absolute_error(y_test, predictions)

# Calculate MAPE (Mean Absolute Percentage Error)
mape = np.mean(np.abs((y_test - predictions) / y_test)) * 100

print(f"RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"MAPE: {mape:.2f}%")
```

RMSE: 8.00
MAE: 7.20
MAPE: 4.18%

```
# Visualize the predictions using a line chart
train = data[:training_data_len].copy()
valid = data[training_data_len:].copy()
valid.loc[:, 'Predictions'] = predictions

plt.figure(figsize=(16,8))
plt.title('LSTM Model Predictions vs Actual Prices')
plt.xlabel('Date', fontsize=18)
plt.ylabel('Close Price USD ($)', fontsize=18)
plt.plot(train['Close'])
plt.plot(valid[['Close', 'Predictions']])
plt.legend(['Train', 'Actual Valid', 'Predictions'], loc='lower right')
plt.show()
```

